Efficient Bootstrapping of Matrices in FHE

Rostin Shokri and Nektarios Georgios Tsoutsos

University of Delaware
{rostinsh, tsoutsos}@udel.edu

Abstract. In recent years, Fully Homomorphic encryption (FHE) has proven to be a practical solution to various privacy preserving applications such as neural network inference, private information retrieval, and genome analysis. Industries have started to utilize FHE to enable private computation of user's sensitive data, to protect user privacy. Out of all FHE applications, deep learning inference has been the most popular field of research among FHE researchers and practitioners, as it is incredibly challenging to do encrypted inference under FHE.
Matrix multiplication is the fundamental operation that is used in deep learning, and is notoriously challenging to implement efficiently in FHE. Many works utilize CKKS, SOTA FHE scheme for deep learning. They introduce novel encoding strategies to enable matrix multiplication, but they require very large evaluation keys, high execution time, and matrix dimension limitations. Fortunately, a recent FHE scheme proposed by Gentry and Lee, called the GL scheme, supports matrix multiplication as a native operation, whilst supporting every operation in CKKS. While very promising, the unique ring structure of the scheme requires prime NTT transforms, 3D DFT encoding of the message, which are not researched enough. Additionally, there is no efficient bootstrapping algorithm introduced by this scheme, as bootstrapping is needed to enable deep computations that is required by large deep learning models. In this work, we introduce the first unique and efficient bootstrapping algorithm of the GL scheme.

Keywords: CKKS · Fully Homomorphic Encryption · Matrix-Native Bootstrapping · Privacy-Preserving Machine Learning · Private Inference

1 Introduction

Privacy-preserving machine learning [18, 27, 29] has received sustained attention in recent years, driven by the widespread deployment of machine learning as a service (MLaaS) [26] and the need to protect sensitive user data during inference. Fully Homomorphic Encryption (FHE) [13] is one of the most compelling cryptographic approaches for this setting, as it enables computation directly on encrypted data without requiring decryption by the service provider. In a typical private inference workflow, a client encrypts its input under its secret key, the

server evaluates the model over ciphertexts, and the client decrypts the prediction locally. This model offers strong confidentiality guarantees, but its practicality depends critically on the efficiency of the underlying FHE scheme and, in particular, on the cost of ciphertext refresh operation call bootstrapping [14].

A fundamental obstacle in all FHE schemes is noise growth. To ensure security, ciphertexts contain structured noise, and this noise increases with every homomorphic operation. After sufficiently deep computation, the accumulated noise makes decryption unreliable unless the ciphertext is refreshed. Bootstrapping addresses this problem by homomorphically evaluating the decryption circuit and producing a new ciphertext with reduced noise and renewed computational depth. As a result, bootstrapping is a central building block for deep homomorphic workloads, including modern neural networks with many layers, nonlinearities, and repeated linear algebra operations. As hardware accelerators and FHE algorithms continue to improve, enabling deeper and more expressive encrypted models, the cost of bootstrapping increasingly determines end-to-end system performance.

Most practical FHE-based private inference (PI) systems [7,21,30] today rely on the CKKS scheme [9], which is widely considered to be the SOTA FHE scheme for PI because it supports approximate arithmetic over real and complex numbers and provides high throughput through packed SIMD-style computation. Accordingly, a large body of work has focused on improving CKKS bootstrapping [3,5], exploring tradeoffs among multiplicative depth, latency, precision, and parallelism. However, CKKS remains fundamentally a vector-oriented scheme. Matrix multiplication must still be realized through nontrivial packing layouts, rotations, and sequences of element wise operations [1, 22]. Since matrix multiplication is the dominant kernel in deep learning, this mismatch between the algebra of the scheme and the structure of the workload continues to impose substantial overhead.

The recently proposed GL scheme [15] introduces a significantly different design point. Instead of treating matrix multiplication as a derived operation over vector slots, GL natively supports encrypted matrix arithmetic through a multivariate ring construction and a Gaussian-integer viewpoint, reducing homomorphic matrix multiplication to structured plaintext modular matrix multiplications [15]. This feature makes GL particularly attractive from a systems and hardware perspective. Because the dominant encrypted linear algebra is reduced to plaintext modular matrix multiplication, GL is a natural candidate for GPU acceleration and future specialized FHE hardware. Its arithmetic aligns well with highly optimized dense linear algebra kernels and with massively parallel architectures equipped with matrix-oriented compute units. Consequently, GL has the potential to serve as a strong foundation for high-throughput private inference systems.

Despite this promise, GL currently lacks an efficient and concrete bootstrapping procedure. While the original GL paper notes that the scheme can in principle be combined with existing bootstrapping methods, no efficient realization specialized to GL is currently available [15]. This gap is critical. Without boot-

strapping, GL remains restricted to shallow computations, which severely limits its practical use in deep private inference workloads where repeated ciphertext refresh is unavoidable. In effect, the absence of bootstrapping is the main barrier preventing GL from becoming a practical alternative to CKKS in encrypted deep learning.

Designing bootstrapping for GL is substantially more challenging than simply transplanting a CKKS construction. The difficulty is algebraic, not merely implementation-wise. CKKS bootstrapping is built around a one-dimensional power-of-two cyclotomic ring, where slot/coefficient transforms, automorphisms, and masking structures are well understood. In contrast, a GL ciphertext slice lives in a non-power-of-two (X, W) ring whose structure is intrinsically multivariate. This changes the semantics of rotations, the organization of coefficients, and the action of sparse secret-key monomials during the decryption circuit. Consequently, standard CKKS bootstrapping techniques do not transfer directly. Any practical GL bootstrapping algorithm must therefore be derived from the slice algebra itself rather than obtained by a straightforward reuse of CKKS machinery.

In this work, we bridge this gap by introducing, to the best of our knowledge, the first efficient bootstrapping framework for the GL scheme. Our approach adapts recent shallow bootstrapping ideas from [6] to the GL setting while preserving the structural advantages that make GL attractive in the first place. Rather than flattening the GL ciphertext into a conventional one-dimensional object, we develop a slice-aware construction that works directly with the (X, W) ring structure. This viewpoint is essential both for correctness and for efficiency: it enables us to align the decryption circuit with the native GL algebra, and retain a path toward practical hardware acceleration.

More broadly, this work connects two directions that have so far developed largely independently: recent advances in shallow and highly parallelizable bootstrapping, and emerging matrix-native FHE schemes designed for efficient linear algebra and hardware acceleration. We argue that this connection is necessary for the next generation of privacy-preserving deep learning systems. Without efficient bootstrapping, matrix-native schemes cannot support deep workloads; without matrix-native schemes, substantial performance is lost in mapping inherently matrix-based computations into vector-oriented encrypted arithmetic. Our goal is to unify these directions and provide the algorithmic foundation needed to make GL practical for encrypted inference.

Our contributions are as follows:

- We formulate, to the best of our knowledge, the first concrete bootstrapping framework for the GL scheme, addressing a central open gap between the promising matrix-native computation model of GL and its practical deployment in deep private inference.
- We show that bootstrapping GL requires a slice-aware treatment of the non-power-of-two (X, W) ring structure, and we identify why a direct reuse of standard CKKS bootstrapping techniques is insufficient in this setting.

– We develop a new GL slice bootstrapping procedure that adapts shallow decryption-in-the-exponent ideas to the GL algebra.
– We provide the first step toward making GL a practical foundation for deep encrypted inference, thereby bridging the current gap between matrix-native FHE design and bootstrapped end-to-end private machine learning systems.

Roadmap: The rest of the paper is organized as follows. In Section 2, we review the necessary background on CKKS, the GL scheme, and existing CKKS bootstrapping methods. In Section 3, we present our GL slice-wise SHIP bootstrapping framework, including our novel partial S2C algorithm, and our unique and efficient half SHIP bootstrapping for a GL slice. Section 4 reports our experimental evaluation and comparisons against CKKS baselines. Section 5 discusses prior work on homomorphic matrix arithmetic and bootstrapping, and positions our contribution relative to CKKS bootstrapping algorithms, and other less usable FHE matrix schemes. Finally, Section 6 concludes the paper.

2 Background

In this section, we briefly review the CKKS and GL schemes and summarize the main paradigms for bootstrapping CKKS ciphertexts. We then highlight why bootstrapping the GL scheme is substantially more challenging than directly reusing existing CKKS constructions.

2.1 CKKS and GL Background

CKKS The CKKS scheme is an RLWE-based fully homomorphic encryption scheme for *approximate* arithmetic over real and complex numbers. It is typically instantiated over the power-of-two cyclotomic ring

$$R_q = \mathbb{Z}_q[X]/(X^N + 1),$$

where N is a power of two and q is the ciphertext modulus. Through the canonical embedding, a plaintext polynomial can encode $N/2$ complex values, enabling SIMD-style packed computation. To represent real or complex values with high precision, CKKS uses a scaling factor Δ, so that plaintext values are mapped to approximate ring elements.

A CKKS ciphertext has the usual RLWE form (b, a), and decryption satisfies

$$b + as \approx m,$$

where s is the secret key and m is the encoded plaintext. Homomorphic addition is component-wise, while homomorphic multiplication produces a higher-degree ciphertext under the enlarged secret-key basis $(1, s, s^2)$. Therefore, multiplication must be followed by *relinearization*, which is a form of key switching that returns the ciphertext to the standard two-component form. In addition, multiplication increases the effective scale, so CKKS uses *rescaling* to reduce the modulus and bring the scale back under control. These operations make CKKS especially well suited for approximate numerical workloads such as neural network inference, where additions and multiplications dominate.

GL scheme The Gentry–Lee (GL) scheme is a recently proposed RLWE-based FHE construction designed for *native matrix arithmetic*. Instead of working in a univariate cyclotomic ring and treating matrices as packed vectors, GL uses a multivariate Gaussian-integer ring

$$R'_q = \mathbb{Z}_q[i][X, Y, W]/(X^n - i,\ Y^n - i,\ \Phi_p(W)),$$

while the secret key is sampled from the smaller ring

$$R_q = \mathbb{Z}_q[i][X, W]/(X^n - i,\ \Phi_p(W)).$$

Under this construction, one plaintext polynomial encodes $\varphi(p)$ matrices of size $n \times n$ by evaluation at roots of unity:

$$m(\zeta_j, \zeta_k, \eta_\ell) = M^{(\ell)}_{j,k},$$

where ζ_j, ζ_k are $4n$-th roots of unity associated with the X- and Y-axes, and η_ℓ is a p-th root of unity associated with the W-axis. This yields a native matrix encoding in which row and column structure are represented directly in the plaintext algebra. Because the W-axis has length $\varphi(p)$, which is generally not a power of two, its transform cannot use the standard radix-2 Cooley–Tukey pipeline and instead must rely on a prime-length transform such as Rader's algorithm [25].

For a simple toy example, if $n = 2$ and $\varphi(p) = 2$ (which means $p = 3$), then a single GL plaintext can encode 2 separate 2×2 complex matrices at once. In this case, each Y-slice is an (X, W)-ciphertext with RLWE ring dimension $2n\varphi(p) = 8$, so one GL ciphertext can be viewed as two slice ciphertexts, each carrying part of the batched matrix data.

Supported homomorphic operations in GL Beyond ciphertext–ciphertext matrix multiplication, the GL scheme naturally supports a broad set of matrix operations through its multivariate ring structure [15]. In particular:

- **Plaintext–ciphertext matrix multiplication:** a ciphertext can be multiplied by a plaintext matrix directly through the trace-based matrix multiplication primitive; with transposed encoding, the same mechanism also supports left multiplication by a plaintext matrix.
- **Addition:** ciphertext addition is component-wise and directly corresponds to matrix addition in the encoded domain.
- **Hadamard multiplication:** entrywise matrix multiplication is realized by ordinary ciphertext multiplication, followed by relinearization.
- **Conjugation:** complex conjugation of each encoded matrix is implemented by a ring automorphism together with key switching.
- **Row rotation:** cyclic row shifts inside each encrypted matrix are induced by automorphisms on the X-variable, followed by key switching.
- **Column rotation:** cyclic column shifts are induced by automorphisms on the Y-variable; since the secret key is independent of Y, this operation does not require key switching.

- **Inter-matrix rotation:** rotations across the batched matrices are induced by automorphisms on the W-variable and correspond to cyclic shifts along the batch dimension.
- **Transpose:** matrix transposition is implemented by permuting the X- and Y-variables, followed by key switching.
- **Conjugate transpose:** Hermitian transposition is obtained through a combined automorphism on X, Y, and W, together with key switching.

These operations are important because they show that GL is not only a scheme for fast encrypted matrix multiplication, but a broader matrix-native homomorphic computation model. As noted in the GL paper, the resulting ciphertext structure can also be combined with existing RLWE-based bootstrapping techniques by applying bootstrapping independently to each Y-slice [15].

A major advantage of GL is that matrix operations arise naturally from the ring structure. Matrix addition and Hadamard multiplication correspond to direct ring operations, while row and column transformations can be expressed through ring automorphisms. Most importantly, encrypted matrix multiplication is realized through a homomorphic trace construction using an auxiliary variable, reducing ciphertext-ciphertext matrix multiplication to four matrix multiplications of ciphertext components followed by key switching. Therefore, GL avoids repeated slot-to-coefficient and coefficient-to-slot conversions and provides a much more natural algebraic substrate for matrix-heavy workloads.

Why GL is different from CKKS for bootstrapping Although GL can be viewed as CKKS-like in the sense that it supports approximate arithmetic and is based on RLWE, its internal algebra is fundamentally different from standard CKKS. In CKKS, bootstrapping is designed around a one-dimensional power-of-two cyclotomic ring, where coefficient/slot transforms, automorphisms, and masking structures are well understood. In contrast, a GL ciphertext slice lives in a non-power-of-two (X, W) ring over $\mathbb{Z}_q[i]$, and the decryption circuit must account for multivariate structure, Gaussian-integer coefficients, and the interaction between the X- and W-axes. Consequently, standard CKKS bootstrapping methods do not transfer directly. This algebraic mismatch is precisely what makes GL bootstrapping both challenging and interesting.

2.2 CKKS Bootstrapping Background

Bootstrapping is the operation that refreshes a ciphertext by reducing its noise and restoring its available modulus budget. For CKKS, the goal is to transform a low-level ciphertext back into a high-level ciphertext that encrypts (approximately) the same plaintext. Existing CKKS bootstrapping methods can be broadly divided into two families.

ModRaise + approximate modular reduction The classical CKKS bootstrapping approach first applies ModRaise to lift a ciphertext from modulus q to a larger modulus Q. After this operation, the ciphertext decrypts to a value of the form

$$m + kq,$$

where m is the target plaintext and kq is an unwanted multiple of the old modulus. The central challenge is therefore to remove the kq term homomorphically. Classical CKKS bootstrapping solves this by evaluating an approximate modular reduction map, usually denoted EvalMod, through polynomial approximation. In practice, this is implemented by transforming the ciphertext between coefficient and slot representations, applying a polynomial approximation to a periodic function related to modular reduction, and then transforming back.

This paradigm underlies the original CKKS bootstrapping algorithm [5] and many subsequent refinements [17, 28]. Improvements in this line of work typically focus on reducing multiplicative depth, improving approximation quality, optimizing the coefficient-to-slot and slot-to-coefficient transforms, or exploiting sparse-secret variants to reduce latency and failure probability.

Homomorphic evaluation of the decryption equation A more recent line of work departs from the classical EvalMod viewpoint and instead refreshes a ciphertext by more directly homomorphically evaluating a decryption-style relation. Rather than approximating modular reduction through sine or Chebyshev-based constructions, these methods embed the relevant low-level coefficients into roots of unity and recover the message through a decryption-in-the-exponent style computation. This viewpoint can substantially reduce multiplicative depth and improve parallelism.

Representative works in this family include SHIP [6] and SPRU [11]. These approaches are especially attractive when low depth and implementation efficiency are critical, since they avoid some of the most expensive components of the classical CKKS bootstrapping pipeline. In particular, SHIP replaces the usual coefficient-to-slot (C2S) stage with a shallower decryption-oriented construction and relies primarily on the S2C. Instead of the ModRaise and C2S operation, SHIP has an plaintext encoding step, where they encode the polynomial components of the ciphertext, and bootstrap using masked-column selection and blind-rotation style primitives.

Bootstrapping viewpoint adopted in this work The bootstrapping method developed in this paper follows the SHIP-style family of constructions. Concretely, our approach adopts the decryption-in-the-exponent viewpoint and leverages S2C-style transformations while avoiding the classical C2S-centered EvalMod pipeline. This choice is motivated by two considerations. First, SHIP provides an efficient and shallow bootstrapping framework that is better aligned with the low-depth requirements of practical implementations. Second, its decryption-oriented structure offers a promising starting point for adapting bootstrapping to the nonstandard slice algebra of the GL scheme.

2.3 Threat Model

We consider the standard honest-but-curious private inference setting. A client encrypts its input under its own private secret key and sends the ciphertexts to an untrusted server, which holds the public FHE keys and performs homomorphic

evaluation, such as bootstrapping, without learning the client's plaintext input or output. The server is assumed to follow the prescribed protocol correctly, but may attempt to infer information from the ciphertexts, evaluation keys, and intermediate bootstrapping material it observes.

CKKS and GL achieve IND-CPA security by instantiating encryption as an RLWE encryption problem: a ciphertext is a noisy affine masking of the plaintext under a secret ring element, so distinguishing encryptions of two chosen messages is computationally hard assuming RLWE is hard over the underlying ring. In CKKS this ring is the standard cyclotomic RLWE ring, while in GL the multivariate Gaussian-integer construction is chosen so that it remains reducible to standard RLWE hardness over an appropriate cyclotomic ring. We do not consider the stronger IND-CPA^D [19] setting, which for approximate schemes such as CKKS and GL typically requires additional countermeasures such as noise flooding [20], although such defenses can be incorporated in a straightforward manner if needed.

3 Framework

3.1 Our Novel Bootstrapping Algorithm

A major advantage of our bootstrapping algorithm is that it requires substantially fewer rotation keys than its CKKS counterpart. In standard CKKS bootstrapping, coefficient/slot transforms are implemented through large families of slot rotations and the corresponding evaluation keys, which creates significant memory overhead for key generation, communication, and storage. In contrast, our GL bootstrapping pipeline exploits the native matrix structure of the scheme. In particular, the X-axis inversion is realized through matrix multiplication and conjugate-transpose operations, while the slice-wise SHIP phase restricts hidden selection to the X-axis only. Thus, the number of required key-switched operations is greatly reduced, which lowers both memory footprint and execution time.

Another important benefit of our construction is its flexibility. After the GL ciphertext is refreshed, the computation does not need to remain in a single homomorphic representation. Depending on the structure of the remaining program, the refreshed ciphertext can continue either in the GL encoding, to exploit native encrypted matrix arithmetic, or in a CKKS-style representation, if subsequent vector-oriented SIMD operations are more efficient. This flexibility is especially useful for large compiler-driven homomorphic programs, where an optimizer can choose when to remain in GL and when to switch encodings to minimize total evaluation cost.

Slots-to-Coeffs Operation in GL In the GL scheme, a plaintext polynomial

$$m(X,Y,W) \in R'_q = \mathbb{Z}_q[i][X,Y,W]/\langle X^n - i,\ Y^n - i,\ \Phi_p(W)\rangle$$

encodes $\varphi(p)$ matrices of size $n \times n$ through the evaluation rule

$$m(\zeta_j, \zeta_k, \eta_\ell) = M^{(\ell)}_{j,k}, \qquad j, k \in [0, n),\ \ell \in [0, \varphi(p)).$$

Thus, the plaintext is naturally in a three-dimensional slot domain along the X, Y, and W axes.

A key structural fact of the GL scheme is that the secret key belongs to the smaller ring

$$R_q = \mathbb{Z}_q[i][X, W]/\langle X^n - i,\ \Phi_p(W)\rangle,$$

so it is independent of Y. Therefore, if we write

$$m(X, Y, W) = \sum_{y=0}^{n-1} m_y(X, W) Y^y,$$

then the ciphertext may be viewed as n RLWE ciphertexts over the inner (X, W)-ring, one for each Y-slice. In other words, the Y-axis acts as an outer batching index, while each slice is an (X, W) ciphertext.

Why the Y-DFT does not need to be inverted homomorphically. Our goal is not to produce a fully coefficient-encoded element of R'_q. Instead, the goal is to prepare the ciphertext for slice-wise bootstrapping by converting each (X, W) slice from slot form back to coefficient form. Since bootstrapping is applied independently to each Y-coefficient ciphertext, there is no need to homomorphically undo the Y-axis transform. The Y-axis can remain as the outer slice index throughout preprocessing.

Accordingly, the S2C procedure in our GL pipeline inverts only the X- and W-axis transforms:

$$\texttt{xyw} \longrightarrow \texttt{yw_only} \longrightarrow \texttt{y_only}.$$

After this process, the message is coefficient-encoded in X and W, while Y still indexes the family of slices. This is precisely the representation required for our slice-wise SHIP-style bootstrapping procedure.

State interpretation. We use the following message-encoding states:

- `xyw`: the message remains in slot form along all three axes;
- `yw_only`: the X-axis transform has been removed, while Y and W remain in slot form;
- `y_only`: both the X- and W-axis transforms have been removed, while Y remains as the outer slice index.

Overall GL S2C pipeline Let C be a GL ciphertext in the standard three-axis slot encoding. The preprocessing stage is

$$\mathrm{S2C}_{\mathrm{GL}}(C) = \mathrm{WInv}\big(\mathrm{XInv}(C)\big).$$

Algorithm 1 Matrix-multiplication-based X-axis inversion

Require: A GL ciphertext C in state `xyw`
Require: Precomputed plaintext transform P_X
Ensure: A ciphertext C in state `yw_only`
1: $C \leftarrow \mathrm{ConjTranspose}(C)$
2: $C \leftarrow \mathrm{MatMul}(C, P_X)$
3: $C \leftarrow \mathrm{ConjTranspose}(C)$
4: $C \leftarrow \mathrm{Rescale}(C)$
5: Update the message state from `xyw` to `yw_only`
6: **return** C

The first stage removes the X-axis transform by reusing the GL plaintext-ciphertext matrix multiplication primitive. The second stage removes the W-axis transform through a diagonal decomposition accelerated with a baby-step/giant-step strategy. Each stage is followed by a rescale, so the implementation starts from a level-2 ciphertext and ends at level 0.

Algorithm 1 Overview. For each fixed W-slot, the payload may be viewed as an $n \times n$ matrix over the (X, Y) indices. The inverse X-axis transform is a linear transform acting on one matrix dimension. Since GL already provides a plaintext-ciphertext matrix multiplication primitive, this transform can be realized by multiplying with a fixed plaintext matrix. The conjugate-transpose trick is used only to express the required left action through the available right-multiplication primitive.

Algorithm 2 Overview. The inverse W-transform is a linear transform across the batched matrix index. A diagonal decomposition expresses this transform as a sum of rotated ciphertexts, each multiplied by a W-diagonal plaintext. The BSGS decomposition groups these terms into blocks. Inside each block, only the short baby rotations are materialized explicitly, while the large giant rotation is applied once to the full block contribution. In the implementation, the expensive decomposition work is hoisted and reused across several terms.

Final output of GL S2C Combining the two stages gives

$$\texttt{xyw} \xrightarrow{\mathrm{XInv}} \texttt{yw_only} \xrightarrow{\mathrm{WInv}} \texttt{y_only}.$$

Thus, the final ciphertext is coefficient-encoded in the inner variables X and W, while Y remains the outer slicing dimension. This is why a separate homomorphic inversion of the Y-DFT is unnecessary in our GL bootstrapping pipeline.

GL Slice SHIP Half-Bootstrapping After the S2C preprocessing, the ciphertext is in state `y_only`. At this point, we first switch the ciphertext from the primary secret key to the sparse low-level bootstrapping secret. This sparse key is the one used by the SHIP-style decryption-in-the-exponent phase. We then

Algorithm 2 BSGS-based W-axis inversion

Require: A GL ciphertext C in state `yw_only`
Require: Precomputed W-diagonal plaintexts $\{D_t\}_{t=0}^{\varphi(p)-1}$
Require: Baby-step size m
Ensure: A ciphertext A in state `y_only`

```
Convert C to a representation where the W-axis is explicit in evaluation form
G ← ⌈φ(p)/m⌉
for j = 0 to m − 1 do
    C_j ← ρ_W^j(C)
end for
A ← 0
for g = 0 to G − 1 do
    T_g ← 0
    for each valid t ∈ {gm, ..., min(φ(p) − 1, gm + m − 1)} do
        j ← t − gm
        T_g ← T_g + C_j ⊙ ρ_W^{−gm}(D_t)
    end for
    A ← A + ρ_W^{gm}(T_g)
end for
A ← Rescale(A)
Update the message state from yw_only to y_only
return A
```

split the ciphertext into Y-slices and bootstrap each slice independently in the inner (X, W)-ring.

For one slice, let the ciphertext be $(B(X, W), A(X, W))$. Since the slice lives in the Gaussian-integer ring $\mathbb{Z}_q[i][X, W]/\langle X^n - i, \Phi_p(W)\rangle$, we write

$$A = A_0 + iA_1, \qquad B = B_0 + iB_1,$$

with A_0, A_1, B_0, B_1 in the real coefficient ring. The sparse slice secret has monomials of the form

$$s_r(X, W) = \sigma_r X^{\alpha_r} W^{\beta_r}, \qquad \sigma_r \in \{\pm 1\}.$$

This induces two real coefficient branches:

$$M_0 = B_0 + A_0 s, \qquad M_1 = B_1 + A_1 s.$$

Rather than attempting to apply SHIP directly to Gaussian exponents, we bootstrap these two real branches separately and recombine them only at the end. This preserves the real-valued scalar reduction framework of SHIP while respecting the GL slice algebra.

For efficiency, we keep the W-axis in the coefficient basis. Multiplication by W^β is represented by a real $\varphi(p) \times \varphi(p)$ matrix B_β, so the hidden W-part of each sparse secret monomial becomes an ordinary linear transform on the W-coefficient vector. This allows the hidden selection problem to remain on the

X-axis only. We then apply a SHIP-style hybrid masked-column procedure: a coarse X-selection is performed by encrypted masked columns, while the fine X-residue is handled by blind rotation through HMuxRot. For efficiency, our implementation uses the gadget-decomposed HMuxRot structure corresponding to Algorithm 5 in SHIP.

To further reduce the number of candidate masked columns, we adopt SHIP's sparse-secret spacing assumption on the flattened slice-monomial index

$$t = \varphi(p)\alpha + \beta.$$

This yields a public interval for each sparse nonzero, which is then projected to a smaller admissible X-window. The masked-column bank is generated only for candidate (j_1, β) pairs consistent with that public interval. In this way, the algorithm preserves the same sparse-secret support model as SHIP while avoiding a full search over all X-positions.

Algorithm 3 Overview. The sparse secret key is first introduced only at the lowest level, in the same spirit as sparse-secret bootstrapping methods for CKKS. Each Y-slice is then processed independently. For each slice, we split the Gaussian ciphertext into two real coefficient branches so that the scalar reduction step remains in the real domain, which is the setting for which SHIP is designed. The W-component of each sparse secret monomial is handled in coefficient form through the precomputed matrices B_β, while the X-component is handled by a hybrid masked-column procedure: a coarse candidate block is selected by encrypted masked columns, and the fine X-offset is selected by HMuxRot. After all sparse monomial contributions are computed, each branch is refreshed through the SHIP product tree and conjugation-add step. Finally, the two refreshed branches are recombined and the Y-slices are merged back into a full GL ciphertext. Now to apply a full bootstrap, we do the GL S2C operation first, then Algorithm 3.

Output representation. The output of the half-bootstrap is a refreshed GL ciphertext obtained by recombining the two real coefficient branches after the SHIP refresh stage. Since the slice-wise procedure preserves the GL slice structure, the resulting ciphertext may continue either in GL form or be converted to a CKKS-style representation depending on the needs of subsequent homomorphic computation.

4 Experiments

All experiments were conducted on an Ubuntu workstation equipped with an AMD 7970X CPU and 160 GB of RAM. We note that all experimental results are based on *single-threaded execution.*

Algorithm 3 GL slice SHIP half-bootstrap

Require: A GL ciphertext C in state `y_only`, under the primary secret key
Require: Sparse low-level bootstrapping key and masked-column / HMuxRot evaluation keys
Ensure: A refreshed GL ciphertext C'

$C \leftarrow \text{KeySwitchToSparse}(C)$
Split C into Y-slices $\{C_y\}_{y=0}^{n-1}$
for each Y-slice $C_y = (B, A)$ **do**
    Split the Gaussian coefficients:

$$A = A_0 + iA_1, \qquad B = B_0 + iB_1$$

    Generate the branchwise SHIP plaintext families from the slice coefficients
    Generate the two branchwise base plaintexts from B_0 and B_1
    for each branch $b \in \{0, 1\}$ **do**
        for each sparse secret monomial $s_r = \sigma_r X^{\alpha_r} W^{\beta_r}$ **do**
            Perform coarse masked-column accumulation over admissible (j_1, β) candidates
            Apply X-only blind rotation / HMuxRot on the fine residue j_0
            Store the resulting branchwise premux ciphertext $\text{PM}_r^{(b)}$
        end for
        Build the SHIP product tree from the branch base plaintext and $\{\text{PM}_r^{(b)}\}_r$
        Apply SHIP conjugation-add postprocessing to obtain the refreshed branch output
        Apply the real S2C transform to return the branch to coefficient form
    end for
    Recombine the two refreshed coefficient branches for slice y:

$$C'_y \leftarrow C'_{y,0} + i\, C'_{y,1}$$

end for
Merge the refreshed Y-slices $\{C'_y\}_{y=0}^{n-1}$ into one GL ciphertext C'
return C'

4.1 Implementation Details

We implemented both the CKKS and GL scheme from scratch in C++. We utilized the hybrid key switching technique from [16], and same rescaling, Modup and ModDown algorithms for both schemes. The NTT algorithms core butterfly operations and radix algorithms are also implemented from scratch in scalar C++. We implemented SHIP CKKS bootstrapping algorithm [6] from scratch (since the original paper did not make their code publicly available), and compare it with their reported numbers to demonstrate comparable performance. Notably, the GL scheme has previously only been implemented in the proprietary Desilo library [12]. Although this library allows users to execute FHE operations via a Python interface, its closed-source nature obscures the internal cryptographic mechanisms. Therefore, evaluating and extending the scheme necessitated developing a custom C++ implementation from scratch.

Table 1. Parameter sets and evaluation results for real-number bootstrapping reported in [6], and our implementation. All experiments are single-threaded. The timings of [6] are copied directly from the paper because the implementation is not open source, so we could not run it ourselves.

Param. set	N	$\log PQ$	dnum	$(\theta, \ell_0/\theta, B)$	# mult depth	[6] (s)	Ours (s)	Speedup
LL13	2^{13}	218	8	(6, 64, 4)	7	3.02	3.26	0.92×
LL14	2^{14}	437	4	(9, 64, 4)	7	4.91	6.09	0.80×
HT14	2^{14}	438	8	(9, 64, 4)	15	13.2	17.03	0.77×
HT15	2^{15}	881	4	(17, 64, 4)	15	29.6	31.01	0.95×

Table 2. Depth-2 S2C comparison between CKKS and GL using the GL paper's reported parameter set. CKKS uses the Cooley-Tukey S2C depth 2 decomposition. Input ciphertexts are encrypted at depth 2. Ring dimension of the CKKS execution is $N = 2n\varphi(p)$. The GL latency is amortized over n slices.

Param. set	n	p	$\varphi(p)$	$2n\varphi(p)$	Δ	CKKS latency (s)	GL latency (amortized) (s)	Speedup
GL-256-17	256	17	16	2^{13}	2^{40}	0.14	0.05	**2.8** ×
GL-512-17	512	17	16	2^{14}	2^{40}	0.27	0.12	**2.25** ×
GL-1024-17	1024	17	16	2^{15}	2^{40}	0.67	0.29	**2.31** ×

4.2 CKKS SHIP Comparison

We compare our CKKS SHIP implementation against the reported results of SHIP in [6]. We report the published SHIP parameter sets and evaluation numbers alongside our own measurements in order to assess how closely our implementation matches their reported performance. For consistency, all latency values in Table 1 are expressed in seconds.

As shown in Table 1, our implementation is competitive with the reported SHIP results across all parameter sets. The measured slowdown ranges from approximately 5% to 29%, with the smallest gaps appearing for LL13 and HT15. We attribute this difference primarily to implementation-level factors, including differences in hardware and the fact that the SHIP implementation uses hand-optimized AVX2 kernels, whereas our implementation is written in scalar C++.

4.3 GL vs CKKS S2C

We compare a depth-2 CKKS S2C baseline against our depth-2 GL S2C implementation. On the CKKS side, the baseline is a two-stage Cooley-Tukey decomposition for S2C, so the comparison is depth-matched. Table 2 shows that our GL S2C consistently outperforms the depth-matched CKKS S2C baseline across all tested parameter sets, with speedups between 2.25× and 2.8×. This improvement comes from exploiting the native GL slice structure. our method performs a dedicated X-axis inversion followed by a structured W-axis inversion, which matches the algebra of the GL ciphertext.

Table 3. GL vs. CKKS complex bootstrapping comparison. For each parameter set, we compare our GL slice-wise SHIP-style bootstrapping against a CKKS SHIP baseline at the matched RLWE ring dimension. We use the same security assumptions as [6]. The GL latency is amortized over n slices.

Param. set	n	p	$\varphi(p)$	$2n\varphi(p)$	Δ	mult depth	CKKS latency (s)	GL latency (amortized) (s)	Speedup
GL-256-17	256	17	16	2^{13}	2^{40}	8	7.34	9.87	0.74 ×
GL-512-17	512	17	16	2^{14}	2^{40}	8	13.42	18.23	0.73 ×
GL-1024-17	1024	17	16	2^{15}	2^{40}	8	34.23	44.98	0.76 ×

4.4 GL vs CKKS Bootstrapping

To compare our novel bootstrapping algorithm with the CKKS bootstrapping SHIP from [6], we execute both algorithms with different parameter sets shown in Table 3.

To ensure a fair comparison, the CKKS SHIP baseline reported in Table 3 is also our own implementation. As the table shows, the current GL slice-wise bootstrap is consistently slower than the matched CKKS SHIP baseline, achieving 0.73×–0.76× of the CKKS speed across the three parameter sets. This gap is expected at the current stage, since our GL pipeline still includes additional NTT transforms, and harder ring structure to compute the decryption phrase, due to the non-power-of-two (X, W) ring, whereas SHIP takes advantage of the easy ring structure of CKKS [6]. Nevertheless, the results are encouraging: our GL implementation remains in the same overall runtime regime, which suggests that further optimization of the GL-specific transforms, plaintext matrix multiplication, and slice-wise key-switching routines could substantially narrow the gap while preserving the matrix-native advantages of the GL scheme [6, 15].

5 Related Works

5.1 Other Matrix Schemes

A recent line of works reduce homomorphic matrix multiplication to a small number of plaintext modular matrix multiplications. The first line of works, Liu and Zhang, and later Bae *et al.*, showed how plaintext-ciphertext matrix multiplication can be reduced to matrix multiplication on ciphertext components and input plaintexts [2, 23]. Park extended this idea to ciphertext–ciphertext matrix multiplication for large matrices [24]. As emphasized in the GL paper, these approaches are attractive because they can leverage highly optimized dense linear algebra libraries, but they also come with important limitations: the matrix dimension is tied to the ring dimension, the matrices are coefficient-encoded rather than slot-encoded, and practical use together with SIMD operations requires repeated C2S and S2C transforms [24]. Compared with GL, these approaches do not provide a native matrix encoding: matrix multiplication is accelerated, but the ciphertext still lives in a coefficient-oriented CKKS representation, so mixing matrix multiplication with ordinary slot-domain SIMD operations incurs repeated C2S and S2C conversions. In addition, their matrix dimensions and batch

structure remain constrained by the underlying ring organization, whereas GL supports slot-encoded matrix arithmetic directly through its multivariate algebra and therefore avoids this representation mismatch. This makes GL the appropriate scheme to accelerate and find new and efficient bootstrapping algorithms for.

A closely related parallel work to GL is the recent CKKS-based matrix scheme [8], which extends reduction-based ciphertext–plaintext and ciphertext–ciphertext matrix multiplication to small and medium matrix dimensions by batching many instances together. Their approach uses a *Slots-in-Coefficient* (SinC) encoding, so that matrix dimensions and batch sizes are no longer tied to the ring dimension, and they report substantial speedups for batched matrix multiplication over prior CKKS-based methods [8]. This is an important advance, since it addresses one of the main limitations of earlier reduction-based matrix methods. However, it still remains fundamentally within a CKKS coefficient-encoding framework. As a result, when a workload must alternate between matrix multiplication and ordinary slot-domain operations such as Hadamard products, nonlinear layers, or standard SIMD arithmetic, one must effectively move between coefficient-oriented and slot-oriented representations, which is expensive. On the other hand, GL makes the matrix-native GL scheme viable for deep computation, and our work providing an efficient bootstrapping algorithm makes it the SOTA scheme to use for private inference.

5.2 CKKS bootstrapping

The classical CKKS bootstrapping line refreshes ciphertexts by combining modulus raising, homomorphic coefficient-to-slot and slot-to-coefficient transforms, and polynomial approximation to an approximate modular reduction function [3–5, 16]. These methods typically require substantial multiplicative depth because they combine homomorphic DFT/iDFT layers with a high-degree polynomial approximation step, which in turn forces large ciphertext moduli and large ring dimensions. SHIP [6] introduced a shallower and more parallelizable alternative by evaluating a decryption-style relation in the exponent, thereby significantly reducing the top-level transform cost and overall bootstrapping depth [6]. This decryption-oriented viewpoint is the main conceptual starting point for our work.

Why direct import of existing CKKS bootstrapping is insufficient for GL The GL paper notes that the scheme is compatible in principle with existing RLWE bootstrapping algorithms [15]. However, importing a standard CKKS bootstrapping method into GL is not satisfactory in practice. First, classical CKKS bootstrapping inherits the high depth of polynomial approximation and repeated homomorphic coefficient/slot transforms, which SHIP was specifically designed to reduce [6].

An example of a polynomial approximation based bootstrapping, that works for non-power-of-two ring in CKKS is the work from [10]. This work was suggested by the GL paper [15], as a starting point to implement bootstrapping

for the GL scheme. The work from [10] generalizes CKKS to a tensor-style packing structure and supports matrix operations together with a multidimensional bootstrapping procedure. Their bootstrapping still follows the classical CKKS paradigm: after a `SubSum` step, it applies multidimensional S2C and C2S linear transforms together with a `RemoveIPart` stage to remove the unwanted modulus term via a polynomial approximation. This remains a transform-heavy CKKS-style bootstrapping method that still requires many ciphertext rotations, constant multiplications, and rotation keys. In contrast, our work derives a bootstrapping procedure directly from the GL slice algebra. Rather than importing a generic multidimensional coefficient/slot pipeline into GL, we bootstrap each Y-slice in its native (X, W) structure using a SHIP-style decryption-oriented construction. We also derived a novel S2C algorithm based on the unique GL 3D DFT and ring structure. Our bootstrapping algorithm is natural for GL, avoids relying on expensive C2S conversion as the main mechanism, and preserves the matrix-native advantages that motivate the GL scheme in the first place.

Our contribution relative to prior work. Our work is complementary to the prior matrix-arithmetic literature and to existing CKKS bootstrapping research. We provide the missing efficient bootstrapping layer for GL itself. This is crucial because, without efficient bootstrapping, GL remains impractical for deep neural networks, despite its strong advantages for encrypted matrix multiplication and hardware acceleration.

6 Conclusion

This work addresses a key missing component in making the Gentry–Lee (GL) scheme practical: efficient bootstrapping. While GL is highly promising for privacy-preserving machine learning due to its native support for encrypted matrix arithmetic and strong potential for hardware acceleration, the absence of a concrete bootstrapping method has limited its applicability to shallow computations.

We showed that bootstrapping GL is substantially more challenging than directly reusing existing CKKS techniques, since GL ciphertext slices live in a non-power-of-two (X, W) ring over Gaussian integers. To address this, we developed a slice-aware bootstrapping framework and adopted the SHIP family of CKKS bootstrapping methods. Our construction adapts the decryption-oriented viewpoint of SHIP to the GL setting while preserving the structural properties that make GL attractive for matrix-heavy workloads.

Overall, this work provides a first step toward bridging the gap between matrix-native FHE schemes and practical deep encrypted inference. Open challenges for future research include exploring more optimal prime NTT implementations like Bluestein's algorithm, and exploring GPU acceleration for the matrix-oriented primitives that dominate GL bootstrapping.

Acknowledgments

This work has been supported by NSF Award #2239334.

References

1. Aikata Aikata and Sujoy Sinha Roy. Secure and efficient outsourced matrix multiplication with homomorphic encryption. In *International Conference on Cryptology in India*, pages 51–74. Springer, 2024.
2. Youngjin Bae, Jung Hee Cheon, Guillaume Hanrot, Jai Hyun Park, and Damien Stehlé. Plaintext-ciphertext matrix multiplication and fhe bootstrapping: fast and fused. In *Annual International Cryptology Conference*, pages 387–421. Springer, 2024.
3. Jean-Philippe Bossuat, Christian Mouchet, Juan Troncoso-Pastoriza, and Jean-Pierre Hubaux. Efficient bootstrapping for approximate homomorphic encryption with non-sparse keys. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 587–617. Springer, 2021.
4. Hao Chen, Ilaria Chillotti, and Yongsoo Song. Improved bootstrapping for approximate homomorphic encryption. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 34–54. Springer, 2019.
5. Jung Hee Cheon, Kyoohyung Han, Andrey Kim, Miran Kim, and Yongsoo Song. Bootstrapping for approximate homomorphic encryption. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 360–384. Springer, 2018.
6. Jung Hee Cheon, Guillaume Hanrot, Jongmin Kim, and Damien Stehlé. Ship: A shallow and highly parallelizable ckks bootstrapping algorithm. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 398–428. Springer, 2025.
7. Jung Hee Cheon, Minsik Kang, Taeseong Kim, Junyoung Jung, and Yongdong Yeo. Batch inference on deep convolutional neural networks with fully homomorphic encryption using channel-by-channel convolutions. *IEEE Transactions on Dependable and Secure Computing*, 22(2):1674–1685, 2024.
8. Jung Hee Cheon, Minsik Kang, and Junho Lee. Fast batch matrix multiplication in ciphertexts. *Cryptology ePrint Archive*, 2025.
9. Jung Hee Cheon, Andrey Kim, Miran Kim, and Yongsoo Song. Homomorphic encryption for arithmetic of approximate numbers. In *International conference on the theory and application of cryptology and information security*, pages 409–437. Springer, 2017.
10. Jung Hee Cheon, Andrey Kim, and Donggeon Yhee. Multi-dimensional packing for heaan for approximate matrix arithmetics. *Cryptology ePrint Archive*, 2018.
11. Jean-Sébastien Coron and Robin Köstler. Low-latency bootstrapping for ckks using roots of unity. *Cryptology ePrint Archive*, 2025.
12. Desilo Inc. The DESILO FHE library. https://fhe.desilo.dev/latest/, 2026.
13. Craig Gentry. Fully homomorphic encryption using ideal lattices. In *Proceedings of the forty-first annual ACM symposium on Theory of computing*, pages 169–178, 2009.
14. Craig Gentry, Shai Halevi, and Nigel P Smart. Better bootstrapping in fully homomorphic encryption. In *International workshop on public key cryptography*, pages 1–16. Springer, 2012.

15. Craig Gentry and Yongwoo Lee. Fully homomorphic encryption for matrix arithmetic. *Cryptology ePrint Archive*, 2025.
16. Kyoohyung Han and Dohyeong Ki. Better bootstrapping for approximate homomorphic encryption. In *Cryptographers' Track at the RSA Conference*, pages 364–390. Springer, 2020.
17. Seonghak Kim, Minji Park, Jaehyung Kim, Taekyung Kim, and Chohong Min. Evalround algorithm in ckks bootstrapping. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 161–187. Springer, 2022.
18. Joon-Woo Lee, HyungChul Kang, Yongwoo Lee, Woosuk Choi, Jieun Eom, Maxim Deryabin, Eunsang Lee, Junghyun Lee, Donghoon Yoo, Young-Sik Kim, et al. Privacy-preserving machine learning with fully homomorphic encryption for deep neural network. *iEEE Access*, 10:30039–30054, 2022.
19. Baiyu Li and Daniele Micciancio. On the security of homomorphic encryption on approximate numbers. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 648–677. Springer, 2021.
20. Baiyu Li, Daniele Micciancio, Mark Schultz-Wu, and Jessica Sorrell. Securing approximate homomorphic encryption using differential privacy. In *Annual International Cryptology Conference*, pages 560–589. Springer, 2022.
21. Long Li, Jianxin Lai, Peng Yuan, Tianxiang Sui, Yan Liu, Qing Zhu, Xiaojing Zhang, Linjie Xiao, Wenguang Chen, and Jingling Xue. Ant-ace: An fhe compiler framework for automating neural network inference. In *Proceedings of the 23rd ACM/IEEE International Symposium on Code Generation and Optimization*, pages 193–208, 2025.
22. Lawrence Lim, Vikas Kalagi, Divyakant Agrawal, and Amr El Abbadi. Tricycle: Private transformer inference with tricyclic encodings. *Cryptology ePrint Archive*, 2025.
23. Jing Liu and Liang Feng Zhang. Privacy-preserving and publicly verifiable matrix multiplication. *IEEE Transactions on Services Computing*, 16(3):2059–2071, 2022.
24. Jai Hyun Park. Ciphertext-ciphertext matrix multiplication: Fast for large matrices. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 153–180. Springer, 2025.
25. Charles M Rader. Discrete fourier transforms when the number of data samples is prime. *Proceedings of the IEEE*, 56(6):1107–1108, 1968.
26. Mauro Ribeiro, Katarina Grolinger, and Miriam AM Capretz. Mlaas: Machine learning as a service. In *2015 IEEE 14th international conference on machine learning and applications (ICMLA)*, pages 896–902. IEEE, 2015.
27. Reza Shokri and Vitaly Shmatikov. Privacy-preserving deep learning. In *Proceedings of the 22nd ACM SIGSAC conference on computer and communications security*, pages 1310–1321, 2015.
28. Zejiu Tan, Junping Wan, Zoe L Jiang, Jingjing Fan, Man Ho Au, and Siu Ming Yiu. High-precision homomorphic modular reduction for ckks bootstrapping. In *Australasian Conference on Information Security and Privacy*, pages 232–251. Springer, 2025.
29. Runhua Xu, Nathalie Baracaldo, and James Joshi. Privacy-preserving machine learning: Methods, challenges and directions. *arXiv preprint arXiv:2108.04417*, 2021.
30. Jiawen Zhang, Xinpeng Yang, Lipeng He, Kejia Chen, Wen-jie Lu, Yinghao Wang, Xiaoyang Hou, Jian Liu, Kui Ren, and Xiaohu Yang. Secure transformer inference made non-interactive. *Cryptology ePrint Archive*, 2024.